Adham Ashraf Saeed

Assignment 1_Part 1 & 2 (Python)

GitHub link for the task

Assignment1

Part 1

1. Ask the user to enter daily sales amounts until a negative number is entered.
2. Store all valid sales values in a list.
3. Create a dictionary where the key is the day number and the value is the sale amount.
4. Calculate total sales, average sales, maximum and minimum sale.
5. Create separate functions for each calculation.
6. Add validation to handle non-numeric inputs gracefully.

Part 1

Ask the user to enter daily sales amounts until a negative number is entered.

Add validation to handle non-numeric inputs gracefully.

Store all valid sales values in a list.

```
sales = []
while True:
    try:
        amount = float(input("Enter Daily Sales Amount: "))
        if amount < 0:
            break
        sales.append(amount)

    except ValueError:
        print("Please Enter a Numeric Value")
```

✓ [1] 9s 126ms

Part 1

Display Results

```
print("Sales Amounts Entered:", sales)
```

✓ [2] 128ms

```
Sales Amounts Entered: [10.0, 20.0, 30.0, 40.0, 50.0]
```

Create a dictionary where the key is the day number and the value is the sale amount.

```
sales_dict = {}
for i in range(len(sales)):
    sales_dict[i + 1] = sales[i]

print("Sales Dictionary:", sales_dict)
```

✓ [3] 129ms

```
Sales Dictionary: {1: 10.0, 2: 20.0, 3: 30.0, 4: 40.0, 5: 50.0}
```

Part 1

Calculate total sales, average sales, maximum and minimum sale.

Create separate functions for each calculation.

```
def calculate_total(sales):
    return sum(sales)


def calculate_average(sales):
    return sum(sales) / len(sales)


def calculate_maximum(sales):
    return max(sales)


def calculate_minimum(sales):
    return min(sales)


def display_sales(sales):
    print("Sales Amounts Entered:", sales)
    print("Total Sales Amount:", calculate_total(sales))
    print("Average Sales Amount:", calculate_average(sales))
    print("Maximum Sales Amount:", calculate_maximum(sales))
    print("Minimum Sales Amount:", calculate_minimum(sales))


display_sales(sales)
```

```
Sales Amounts Entered: [10.0, 20.0, 30.0, 40.0, 50.0]
Total Sales Amount: 150.0
Average Sales Amount: 30.0
Maximum Sales Amount: 50.0
Minimum Sales Amount: 10.0
```

Part 2

Here is a sample line of code that can be executed in Python:

```
print("Hello, World!")
```

You can just as easily store a string as a variable and then print it to stdout:

```
my_string = "Hello, World!"
print(my_string)
```

The above code will print `Hello, World!` on your screen. Try it yourself in the editor below!

Input Format

You do not need to read any input in this challenge.

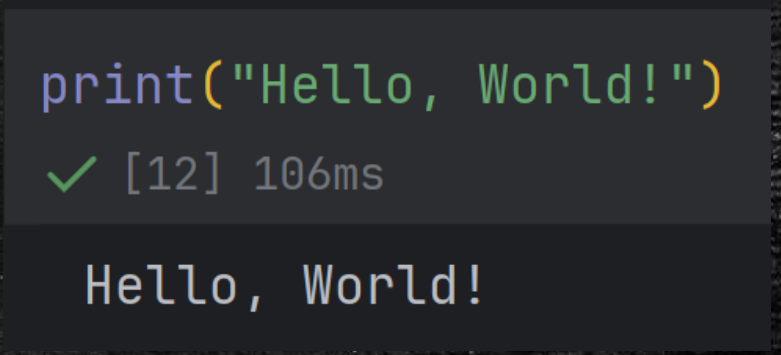
Output Format

Print `Hello, World!` to stdout.

Sample Output 0

```
Hello, World!
```





Part 2

Task

The provided code stub reads two integers from STDIN, a and b. Add code to print three lines where:

1. The first line contains the sum of the two numbers.
2. The second line contains the difference of the two numbers (first - second).
3. The third line contains the product of the two numbers.

Example

$a = 3$

$b = 5$

Print the following:

```
8
-2
15
```



```python
a = int(input("Enter A: "))
b = int(input("Enter B: "))
print(f"A = {a}\nB = {b}")
print(a+b)
print(a-b)
print(a*b)
```

✓ [8] 2s 494ms

```
A = 3
B = 5
8
-2
15
```

Part 2

Task

The provided code stub reads two integers, a and b, from STDIN.

Add logic to print two lines. The first line should contain the result of integer division, a // b. The second line should contain the result of float division, a / b.

No rounding or formatting is necessary.

Example

$a = 3$

$b = 5$

- The result of the integer division $3//5 = 0$.
- The result of the float division is $3/5 = 0.6$.

Print:

```
0
0.6
```



```python
a = float(input("Enter A: "))
b = float(input("Enter B: "))
print(f"A = {a}\nB = {b}")
print(a//b)
print(a/b)
```

✓ [10] 3s 317ms

```
A = 3.0
B = 5.0
0.0
0.6
```

Part 2

Task

Given an integer, n, perform the following conditional actions:

- If n is odd, print `Weird`
- If n is even and in the inclusive range of 2 to 5, print `Not Weird`
- If n is even and in the inclusive range of 6 to 20, print `Weird`
- If n is even and greater than 20, print `Not Weird`

Input Format

A single line containing a positive integer, n.

Constraints

- $1 \leq n \leq 100$

Output Format

Print `Weird` if the number is weird. Otherwise, print `Not Weird`.

Sample Input 0

```
3
```

Sample Output 0

```
Weird
```



```python
n = int(input("Enter a number: "))
print("N =", n)
if n % 2 == 1:
    print("Weird")
elif n % 2 == 0 and 2 <= n <= 5:
    print("Not Weird")
elif n % 2 == 0 and 6 <= n <= 20:
    print("Weird")
elif n % 2 == 0 and n > 20:
    print("Not Weird")
```

✓ [15] 1s 700ms

```
N = 3
Weird
```

Part 2

Task

The provided code stub reads an integer, n, from STDIN. For all non-negative integers $i < n$, print i^2.

Example

$n = 3$

The list of non-negative integers that are less than $n = 3$ is $[0, 1, 2]$. Print the square of each number on a separate line.

```
0
1
4
```



```
n = int(input("Enter N: "))
for i in range(n):
    print(i**2)
```

✓ [11] 2s 87ms

```
0
1
4
```

Part 2

Task

Given a year, determine whether it is a leap year. If it is a leap year, return the Boolean `True`, otherwise return `False`.

Note that the code stub provided reads from STDIN and passes arguments to the `is_leap` function. It is only necessary to complete the `is_leap` function.

Input Format

Read $year$, the year to test.

Constraints

$1900 \leq year \leq 10^5$

Output Format

The function must return a Boolean value (True/False). Output is handled by the provided code stub.

Sample Input 0

```
1990
```

Sample Output 0

```
False
```

Explanation 0

1990 is not a multiple of 4 hence it's not a leap year.



```python
year = int(input("Enter Year: "))
print("Year =", year)
def is_leap(year):
    if year % 400 == 0:
        return True
    elif year % 100 == 0:
        return False
    elif year % 4 == 0:
        return True
    else:
        return False

print(is_leap(year))
```

✓ [16] 2s 939ms

```
Year = 1990
False
```